

Technical-Lead Guide: Deploying ZZedc for a Clinical Trial

End-to-end runbook for the statistician/IT-adjacent team member

ZZedc Team

2026-04-29

Contents

About this guide	2
Chapter 1. Architecture overview	3
The five-layer model	3
Three foundational guarantees	3
Multi-site model	3
Chapter 2. Picking the right backend	4
Decision tree	4
Backend cheat sheet	4
Chapter 3. Installing ZZedc	5
3.1 Prerequisites	5
3.2 Installing from CRAN	5
3.3 Reproducibility: pin dependencies with renv	5
3.4 Verify the install	6
Chapter 4. Configuring the trial	6
4.1 The trial's <code>config.yml</code>	6
4.2 Environment variables (secrets)	6
4.3 Encryption-key custody	7
4.4 Per-trial filesystem layout	7
Chapter 5. Initialising the database	7
5.1 Create the encrypted database	7
5.2 Initialise the audit log	8
5.3 Create the canonical core tables	8
Chapter 6. Loading the content author's workbook	8
6.1 Authentication to Google Sheets	9
6.2 Run the import	9
6.3 Resolve skipped rules	9
6.4 CSV authoring path (alternative to Google Sheets)	9
Chapter 7. Multi-site setup	10
7.1 Verifying site-based access	10
7.2 Adding a site after go-live	10
7.3 Renaming a site	10

Chapter 8. Deploying the application	11
8.1 Quick local launch (development only)	11
8.2 Production pattern: Docker on a single VM	11
8.3 Production pattern: AWS	12
8.4 Other patterns	12
Chapter 9. Backups and disaster recovery	12
9.1 The recovery objective	12
9.2 SQLite/DuckDB daily backup	12
9.3 Restore drill	13
9.4 Key rotation	13
9.5 Full DR procedure	13
Chapter 10. Audit-log verification	14
10.1 Run the integrity check	14
10.2 The MIGRATION_AUDIT_GAP marker	14
Chapter 11. Pre-go-live checklist	14
Infrastructure	14
Database	15
Content	15
Operations	15
People	15
Trial-specific	15
Chapter 12. Day-2 operations	15
Daily (week 1)	15
Weekly (week 2 onward)	16
Monthly	16
Quarterly	16
When to call the project maintainer	16
Appendix A. Migrating from REDCap	16
Appendix B. Per-backend cheat sheets	17
Appendix C. Where the regulatory claims live in the code	17
Companion guides and references	18

About this guide

This is the canonical guide for the **technical lead** of a clinical trial running ZZedc: the statistician, biomedical informaticist, or IT-adjacent colleague who is responsible for installation, server configuration, encryption, backups, and day-2 operations. You work alongside a **content author** (a study coordinator, project manager, or PI's research assistant) who provides the case-report forms, validation rules, and user roster. That person follows Content-Author Guide.

The guide is written for a first small-to-medium clinical trial (10 to 200 participants, 1 to 5 sites). It assumes you have moderate R familiarity (you can `install.packages()`, read a function signature, debug a stack trace) and basic sysadmin skills (you can SSH to a Linux VM, edit a config file, manage TLS certificates). If this describes your intended role exactly, this guide will take you from a fresh CRAN install to a production-ready deployment.

By the end of this guide:

- a server is provisioned with the chosen database backend
 - ZZedc is installed, configured, and reachable over HTTPS
 - the content author’s workbook has been imported
 - multi-site role-based access works
 - the audit log validates end-to-end
 - daily backups run automatically and a restore drill has succeeded
 - the trial is ready for go-live
-

Chapter 1. Architecture overview

ZZedc separates concerns into five layers; each is documented in its own reference vignette, but you should understand the whole picture before deploying.

The five-layer model

+-----+	
1. Shiny application (UI + reactive logic)	
+-----+	
2. Module layer (auth, EDC, reports, exports, compliance)	
+-----+	
3. R6 DatabaseAdapter (dialect-aware queries, pooling)	
+-----+	
4. SQLCIPHER / DBI driver (encryption + transport)	
+-----+	
5. Storage backend	
SQLite DuckDB PostgreSQL MySQL ClickHouse	
+-----+	

The application interacts only with the adapter; backend choice is configured per-project in `config.yml` and is otherwise transparent.

Three foundational guarantees

ZZedc commits to three things and the rest of this guide is about preserving them:

1. **Encryption at rest.** All subject data is stored on disk under AES-256 (SQLCIPHER for the SQLite-and-DuckDB family; server-side TDE for PostgreSQL/MySQL/ClickHouse).
2. **Hash-chained audit log.** Every database mutation appends a row to `audit_log` whose `audit_hash` depends on the previous row’s hash. Tampering with any row breaks the chain. `verify_audit_log_integrity()` validates the chain end-to-end. This claim is what the trial’s 21 CFR Part 11 continuity-of-record argument rests on.
3. **No side channels.** The application never writes subject data to disk except through the adapter; never to stdout/stderr, never to log files, never to caches. If you see subject data on disk outside the encrypted database, something is wrong.

The audit chain in particular is the differentiator vs. less formal capture systems. Treat it as the system’s most load-bearing invariant. Any operation that bypasses the adapter is a bug.

Multi-site model

ZZedc treats *site* as a tag on every user, every subject, and every audit-log entry. The `site_id` column is filterable in every query. Coordinator-class roles are scoped to their site; PI-class roles can be scoped to a site or to all sites; the data-manager and central monitor roles are scoped to all sites by convention.

This model is **logical, not physical**: there is one server, one database, and the application enforces site-based access at the adapter layer. There is no federated multi-database mode and no cross-server replication for site isolation. Trials whose threat model requires physical isolation per site should run a separate ZZedc instance per site.

Chapter 2. Picking the right backend

ZZedc supports five backends. For a first small-to-medium trial, only two are likely candidates: **SQLite** (if you expect under ~50 concurrent users and the database fits on one disk) or **PostgreSQL** (if you need more than that, or have institutional infrastructure already). The other three are for specific situations.

Decision tree

Are you running locally on one machine or one VM?

+++ Yes: SQLite.

+++ No: continue.

Is institutional MySQL/MariaDB already provisioned and managed by your IT team?

+++ Yes: MySQL/MariaDB.

+++ No: continue.

Are you migrating from REDCap?

+++ Yes: MySQL/MariaDB (matches REDCap's native backend; the migration importer is fastest against this).

+++ No: continue.

Will analysts run heavy ad-hoc OLAP queries against the EDC database directly, or do you ship to Parquet/SAS/S3 for analysis anyway?

+++ Heavy OLAP: DuckDB.

+++ Ship for analysis: continue.

Will you have more than 1000 subjects with frequent high-cardinality time-series data (mobile-health, sensor streams)?

+++ Yes: ClickHouse.

+++ No: PostgreSQL.

For 90% of academic clinical trials, the answer is **SQLite** for solo or small-site work and **PostgreSQL** for institutional multi-site deployments. The five backend quickstarts are at:

- `vignette('quickstart-sqlite')`
- `vignette('quickstart-postgresql')`
- `vignette('quickstart-mysql')`
- `vignette('quickstart-duckdb')`
- `vignette('quickstart-clickhouse')`

The rest of this guide assumes SQLite for examples (because it is the simplest and the path most readers will follow); where the answer differs for PostgreSQL/MySQL we call out the delta.

Backend cheat sheet

Backend	When to use	Concurrent users	Encryption	Backup model
SQLite + SQLCipher	Solo, small site, on-prem	up to ~50	SQLCipher	File copy
DuckDB	Analytics-heavy, columnar	up to ~50	At-rest via OS	File copy
PostgreSQL	Institutional, multi-site	hundreds	TDE / pgcrypto	pg_dump or WAL archive
MySQL/MariaDB	REDCap shops, institutional	hundreds	TDE	mysqldump or binlog
ClickHouse	High-volume analytics	thousands	TDE	clickhouse-backup

Chapter 3. Installing ZZedc

3.1 Prerequisites

- R 4.1.0 or later. R 4.4 is recommended.
- A Linux VM (Ubuntu 22.04 LTS or Debian 12 are tested), a macOS machine, or a Windows server.
- ~2 GB disk space (R, dependencies, encrypted database).
- Outbound HTTPS to CRAN to install R packages.
- For a server deployment: a domain name and a TLS certificate.

For a Docker-based deployment (recommended for any deployment above one user), also: Docker 24+ and Docker Compose 2+.

3.2 Installing from CRAN

Once the package is on CRAN, installation is one line:

```
install.packages('zzedc')
```

This pulls all required dependencies, including `shiny`, `bslib`, `DBI`, `RSQLite`, `R6`, `digest`, `openssl`, `jsonlite`, and `googlesheets4`.

For backends other than SQLite, install the optional driver:

```
install.packages('RPostgres') # PostgreSQL
install.packages('RMariaDB')  # MySQL / MariaDB
install.packages('duckdb')     # DuckDB
# ClickHouse: see vignette('quickstart-clickhouse')
```

3.3 Reproducibility: pin dependencies with renv

For any deployment that will outlive a single session, pin dependencies with `renv`. This guarantees that the exact version of every package on the production server matches the version your tests passed against.

```
install.packages('renv')
renv::init()
renv::snapshot()
```

Commit `renv.lock` and `renv/activate.R` to your trial's configuration repository (separate from the package source). On the production server, `renv::restore()` rebuilds the exact library.

3.4 Verify the install

```
library(zzedc)
packageVersion('zzedc')
```

If you see the package version without errors, the install is working. Continue to Chapter 4 for configuration.

Chapter 4. Configuring the trial

ZZedc reads configuration from two places: a YAML file (`config.yml`) and environment variables. The YAML file holds the per-trial settings; environment variables hold secrets.

4.1 The trial's `config.yml`

Create `config.yml` in your trial's working directory. A minimal configuration for a SQLite-backed solo trial:

```
default:
  study_name: "Memory Study 001"
  protocol_id: "MEMORY-001"
  pi_name: "Jane Smith, MD"
  pi_email: "jsmith@example.org"
  study_phase: "II"
  target_enrollment: 60

  db_backend: "sqlite"
  db_path: "/srv/zzedc/MEMORY-001/study.db"

  encryption_at_rest: true
  audit_logging: true
  session_timeout_minutes: 30
```

For PostgreSQL replace the `db_path` block with:

```
db_backend: "postgresql"
db_host: "postgres.internal"
db_port: 5432
db_name: "zzedc_memory001"
db_user: "zzedc_app"
db_sslmode: "require"
```

(The password comes from an environment variable, not the file; see §4.2.)

For MySQL/MariaDB:

```
db_backend: "mysql"
db_host: "mariadb.internal"
db_port: 3306
db_name: "zzedc_memory001"
db_user: "zzedc_app"
```

The full list of options is documented in `?zzedc::read_config`.

4.2 Environment variables (secrets)

Never put secrets in `config.yml`. Use environment variables:

```
# /etc/zzedc/MEMORY-001.env (chmod 600, owned by zzedc user)
DB_ENCRYPTION_KEY="..." # SQLite/DuckDB master key
DB_PASSWORD="..." # Postgres/MySQL/ClickHouse
ZZEDC_SALT="..." # Random 32-byte hex string
ZZEDC_DB_PATH="/srv/zzedc/MEMORY-001/study.db"
GOOGLE_APPLICATION_CREDENTIALS="/etc/zzedc/gcs-svc.json"
```

Source this file in the launch script. For a systemd-managed deployment, use `EnvironmentFile=`. For Docker, use `--env-file`.

4.3 Encryption-key custody

For SQLite/DuckDB, the encryption key is generated at database init. *Where you store it determines the trial's threat model*. Three patterns:

Pattern	Custodian	When to use
Server env var	The server's local environment	Solo work, low-risk trials
AWS KMS / GCP KMS	A cloud key-management service	Most institutional deployments
HSM / Yubikey	Hardware security module	Pharma trials, FDA-submitted studies

For institutional deployments use AWS KMS (or your cloud's equivalent). The `R/aws_kms_utils.R` helpers expose the encrypt/decrypt operations the application needs; see `?zzedc::aws_kms_encrypt` and the Encryption-at-Rest reference vignette.

4.4 Per-trial filesystem layout

Adopt this layout per trial; it makes backups, audits, and restores predictable:

```
/srv/zzedc/
+-- MEMORY-001/
|   +-- study.db           # encrypted SQLite database
|   +-- config.yml        # per-trial config
|   +-- renv.lock         # pinned dependencies
|   +-- exports/          # signed exports
|   +-- backups/          # daily snapshots
|   +-- logs/             # application logs (no PHI)
+-- MEMORY-002/
+-- ...
```

For PostgreSQL/MySQL deployments, `study.db` is replaced by the database service's data directory; the rest of the layout is identical.

Chapter 5. Initialising the database

5.1 Create the encrypted database

For SQLite/DuckDB:

```
library(zzedc)
init <- initialize_encrypted_database(
  db_path = '/srv/zzedc/MEMORY-001/study.db',
```

```

  overwrite = FALSE
)
init$success      # TRUE
init$key          # SAVE THIS; treat as a password

```

The returned `init$key` is the master encryption key. Store it immediately in your secrets manager (or paste it into the trial's env file under `DB_ENCRYPTION_KEY`). If you lose this key, the database is unrecoverable. There is no backdoor.

For PostgreSQL/MySQL the equivalent is to provision an empty database and grant the application user `CREATE` and `INSERT` on it. See the per-backend quickstart vignettes for SQL.

5.2 Initialise the audit log

```

audit_res <- init_audit_logging(
  db_path = '/srv/zzedc/MEMORY-001/study.db'
)
audit_res$success      # TRUE

```

This creates the `audit_log` and `audit_chain` tables. Every subsequent database write will append to them.

5.3 Create the canonical core tables

The setup wizard normally does this. For a programmatic setup, provision the core tables (`study_info`, `edc_users`, `edc_roles`, `subjects`, `form_data`, etc.) via:

```

result <- create_wizard_database(
  config_list = list(
    study_name      = 'Memory Study 001',
    protocol_id     = 'MEMORY-001',
    pi_name         = 'Jane Smith, MD',
    pi_email        = 'jsmith@example.org',
    study_phase     = 'II',
    target_enrollment = 60,
    admin_username  = 'admin',
    admin_password  = 'TempPass!2026-CHANGEME',
    admin_fullname  = 'Trial Admin',
    admin_email     = 'admin@example.org',
    security_salt   = Sys.getenv('ZZEDC_SALT')
  ),
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  backend = 'sqlite'
)

```

This produces an admin user (whose password the admin must change on first login) and the empty `study_info` / `edc_roles` rows the application expects.

Chapter 6. Loading the content author's workbook

The content author follows Content-Author Guide to produce a Google Sheets workbook with three tabs: `Study_Users`, `Data_Dictionary`, `validation_rules`. They share the workbook URL with you. Your job is one R command.

6.1 Authentication to Google Sheets

Authenticate `googlesheets4` once per server:

```
library(googlesheets4)
gs4_auth(email = 'svc-zzedc@example.iam.gserviceaccount.com')
```

Use a service account, not a personal Google account, so the import does not depend on any one human's credentials. The service-account JSON goes into `GOOGLE_APPLICATION_CREDENTIALS` (§4.2).

6.2 Run the import

```
library(zzedc)
result <- setup_zzedc_from_gsheets(
  sheet_id = 'https://docs.google.com/spreadsheets/d/SHEET_ID/edit',
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  imported_by = 'tech_lead'
)

result$success                # TRUE
result$users$imported         # number of users created
result$forms$imported_forms   # number of CRFs created
result$forms$imported_fields  # number of fields across CRFs
result$validation_rules$imported # number of rules
length(result$skipped_rules)  # rules that need manual review
```

The import is idempotent: re-running it after the content author updates the workbook re-applies the deltas. The audit log captures every change.

6.3 Resolve skipped rules

If `length(result$skipped_rules) > 0`, the importer found a validation rule it could not parse. Print the list:

```
str(result$skipped_rules)
```

Each entry is a named list with `field`, `reason`, and the original `rule_dsl`. Send this list to the content author with a note: rewrite each rule using one of the six patterns in Content-Author Guide §3.6, or flag it for manual implementation by you.

6.4 CSV authoring path (alternative to Google Sheets)

For sites that block Google Drive, the content author can deliver three CSV files (`Study_Users.csv`, `Data_Dictionary.csv`, `validation_rules.csv`) instead. Use:

```
result <- setup_zzedc_from_csv(
  csv_dir = '/srv/zzedc/MEMORY-001/incoming/',
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  imported_by = 'tech_lead'
)
```

The contract is otherwise identical. See `vignette('demonstration-trial-setup')` for a full worked example.

Chapter 7. Multi-site setup

Multi-site setup is enforced at the database adapter layer through the `site_id` column on `edc_users`, `subjects`, and `form_data`. The content author has already populated `site_id` on the user roster (Chapter 6 of their guide). Your job is to confirm:

1. Every site appears in the application's *Admin* → *Sites* view.
2. Coordinator-class users see only their site.
3. PI-class users with `site_id = "ALL"` see all sites.

7.1 Verifying site-based access

The cleanest verification is to log in as a coordinator at each site and confirm they cannot list subjects from another site. Do this with synthetic subjects (the content author will create one per site during the dry run; see their Chapter 4).

You can also verify directly against the database:

```
library(DBI)
conn <- connect_encrypted_db(
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  key      = Sys.getenv('DB_ENCRYPTION_KEY')
)
on.exit(DBI::dbDisconnect(conn), add = TRUE)

# How many sites are configured
DBI::dbGetQuery(conn, "
  SELECT site_id, COUNT(*) AS n_users
  FROM edc_users
  GROUP BY site_id
")

# Per-site user list
DBI::dbGetQuery(conn, "
  SELECT site_id, username, role
  FROM edc_users
  ORDER BY site_id, username
")
```

7.2 Adding a site after go-live

To add a site after the trial is live, ask the content author to append rows to the `Study_Users` workbook for the new site, then re-run `setup_zzedc_from_gsheets()`. The importer treats new rows as additions and leaves existing rows alone unless their data has changed.

7.3 Renaming a site

Don't. A `site_id` is referenced by every subject, every form-data row, and every audit-log entry for that site. Renaming would either break those references or require a data-migration step that mutates the audit log (which would break the hash chain). If you must, treat it as a new site (append) and freeze the old site (set `active = FALSE`).

Chapter 8. Deploying the application

8.1 Quick local launch (development only)

For a quick test on your laptop:

```
library(zzedc)
launch_zzedc()
# Open http://localhost:3838
```

This is appropriate for the content author's dry-run protocol (their Chapter 4) but not for any real-site use. For production, use one of the patterns below.

8.2 Production pattern: Docker on a single VM

Recommended for any multi-site deployment. The Dockerfile is included in the package; here is a minimal launch flow.

Build the image

```
docker build -t zzedc:1.0.0 .
```

Compose file

```
# /srv/zzedc/MEMORY-001/docker-compose.yml
services:
  zzedc:
    image: zzedc:1.0.0
    container_name: zzedc-memory001
    restart: unless-stopped
    env_file: /etc/zzedc/MEMORY-001.env
    volumes:
      - /srv/zzedc/MEMORY-001:/srv/zzedc/MEMORY-001
    ports:
      - "127.0.0.1:3838:3838"

  caddy:
    image: caddy:2
    container_name: zzedc-tls
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /etc/zzedc/Caddyfile:/etc/caddy/Caddyfile
      - caddy_data:/data
      - caddy_config:/config

volumes:
  caddy_data:
  caddy_config:
```

Caddyfile (TLS termination)

```
edc.example.org {
```

```

reverse_proxy 127.0.0.1:3838
encode gzip
log {
    output file /var/log/caddy/edc.access.log
}
}

```

Bring it up

```

cd /srv/zzedc/MEMORY-001
docker compose up -d
docker compose ps

```

Caddy obtains a TLS certificate automatically via Let's Encrypt. Confirm <https://edc.example.org> returns the ZZedc login page.

8.3 Production pattern: AWS

For AWS-hosted deployments, see `vignette('quick-start-aws-devops')`. That vignette covers CloudFormation templates, RDS for PostgreSQL, KMS-managed encryption keys, and Application Load Balancer with ACM-managed TLS. The architecture in this guide (Docker on a VM) is appropriate up to several hundred concurrent users; AWS multi-AZ becomes worth the operational overhead beyond that or for trials with FDA submission paths that require documented redundancy.

8.4 Other patterns

- **Posit Connect / shinyapps.io.** Possible for dev/test, but inappropriate for production storage of subject data unless you have a BAA with the host (shinyapps.io does not offer one at the time of writing).
- **Kubernetes.** See `quick-start-aws-devops` §“Kubernetes Deployment”. Overkill for a small trial; appropriate for an organisation running many trials.
- **On-prem bare metal.** Same Docker/Compose pattern, no cloud. Adjust backups (§9) to write to local NAS instead of S3.

Chapter 9. Backups and disaster recovery

9.1 The recovery objective

Your trial's protocol or DMP should specify two numbers:

- **RPO** (Recovery Point Objective): how much data loss is acceptable. For most academic trials: 24 hours.
- **RTO** (Recovery Time Objective): how long to restore service. For most academic trials: 4 hours.

A daily backup with on-site retention satisfies a 24-hour RPO and a 4-hour RTO for a SQLite-backed trial. PostgreSQL can do better via WAL archiving (5-minute RPO).

9.2 SQLite/DuckDB daily backup

```

#!/bin/bash
# /srv/zzedc/MEMORY-001/bin/daily-backup.sh
set -euo pipefail

```

```

TRIAL=MEMORY-001
SRC=/srv/zzedc/${TRIAL}/study.db
DST=/srv/zzedc/${TRIAL}/backups/study-$(date +%F).db

cp "$SRC" "$DST"
gzip "$DST"
sha256sum "${DST}.gz" > "${DST}.gz.sha256"

# Retention: keep 30 daily, 12 monthly
find /srv/zzedc/${TRIAL}/backups/ -name 'study-*.db.gz' \
    -mtime +30 -delete

```

Schedule via cron (0 2 * * * /srv/zzedc/MEMORY-001/bin/daily-backup.sh). Mirror the backups/ directory to S3 or another off-site location.

9.3 Restore drill

Once a quarter, do a real restore drill:

```

# Stop the application
docker compose -f /srv/zzedc/MEMORY-001/docker-compose.yml stop zzedc

# Restore latest backup to a scratch location
TRIAL=MEMORY-001
LATEST=$(ls -lt /srv/zzedc/${TRIAL}/backups/study-*.db.gz | head -1)
gunzip -c "$LATEST" > /tmp/restored-${TRIAL}.db

# Verify the restored DB opens and the audit chain validates
Rscript -e '
  library(zzedc)
  res <- verify_audit_log_integrity(
    db_path = "/tmp/restored-MEMORY-001.db",
    key      = Sys.getenv("DB_ENCRYPTION_KEY")
  )
  stopifnot(isTRUE(res$valid))
'

# Bring the application back
docker compose -f /srv/zzedc/MEMORY-001/docker-compose.yml start zzedc
rm /tmp/restored-MEMORY-001.db

```

Document the drill in the trial’s DMP. An untested backup is not a backup.

9.4 Key rotation

Rotate the encryption key once a year, or sooner if you suspect compromise. The procedure is in vignette('feature-encryption-at-rest') §“Key Rotation”. Plan a 30-minute maintenance window; the trial is unavailable during rotation.

9.5 Full DR procedure

For the four scenarios (database corruption, hardware failure, accidental deletion, full server loss), see the dedicated Disaster Recovery vignette. Each scenario has a documented runbook; familiarise yourself before go-live, not after an incident.

Chapter 10. Audit-log verification

The audit log is the trial’s regulatory backbone. Verify it weekly and after every restore.

10.1 Run the integrity check

```
library(zzedc)
res <- verify_audit_log_integrity(
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  key      = Sys.getenv('DB_ENCRYPTION_KEY')
)

res$valid          # TRUE if chain validates end-to-end
res$first_audit_id # earliest audit_id in the chain
res$last_audit_id  # latest
res$chain_length   # total entries
res$tampered_entries # empty if chain valid
```

If `valid` is `TRUE`, every row in `audit_log` chains to its predecessor and no entry has been tampered with since it was written. If `valid` is `FALSE`, `tampered_entries` lists the rows where the chain breaks. Treat any chain break as a serious incident: stop new writes, capture the database state, and contact your compliance lead before continuing.

10.2 The MIGRATION_AUDIT_GAP marker

If you ran a REDCap migration via `import_redcap_to_zzedc_db(source = "api")` and the source REDCap instance returned a partial or empty audit log (because the API token lacked the `Logging` right), the importer prepends a `MIGRATION_AUDIT_GAP` marker to the chain. This marker is intentional: the chain still validates end-to-end, but continuity-of-record is preserved only from the marker forward. Inspect:

```
library(DBI)
conn <- connect_encrypted_db(
  db_path = '/srv/zzedc/MEMORY-001/study.db',
  key      = Sys.getenv('DB_ENCRYPTION_KEY')
)
DBI::dbGetQuery(conn,
  "SELECT timestamp, details FROM audit_log
   WHERE event_type = 'MIGRATION_AUDIT_GAP'"
)
DBI::dbDisconnect(conn)
```

If the marker is present, document it in the DMP: the trial’s 21 CFR Part 11 continuity argument is anchored at the marker, not at the source-side genesis.

Chapter 11. Pre-go-live checklist

Use this checklist 7 to 14 days before the planned go-live date.

Infrastructure

- ☐ Server provisioned with the chosen backend (Chapter 2)
- ☐ R 4.4+ installed
- ☐ `zzedc` installed from CRAN (or a pinned tarball)
- ☐ `renv.lock` committed to the trial’s config repository

- ☐ config.yml configured for the trial (Chapter 4)
- ☐ Environment variables set (DB_ENCRYPTION_KEY, DB_PASSWORD if applicable, ZZEDC_SALT)
- ☐ TLS certificate in place; HTTPS redirects HTTP

Database

- ☐ Encrypted database initialised
- ☐ Audit log initialised
- ☐ Core tables created via `create_wizard_database()` or the web wizard
- ☐ Admin user created; admin password changed at first login
- ☐ Encryption key stored in the agreed custodian (§4.3)

Content

- ☐ Workbook (or CSV folder) imported via `setup_zzedc_from_gsheets()` / `setup_zzedc_from_csv()`
- ☐ No skipped rules outstanding (or each skipped rule manually addressed)
- ☐ Site list verified (Chapter 7)
- ☐ Site-based access manually verified per site

Operations

- ☐ Daily backup cron scheduled and running
- ☐ Restore drill executed successfully
- ☐ Audit-log integrity check returns `valid = TRUE`
- ☐ Monitoring / alerting in place (disk space, container health, backup success/failure)
- ☐ Trial DMP updated with: server location, custodian, RPO/RTO, encryption-key custody, escalation path

People

- ☐ Content author has run the dry-run protocol (their Chapter 4)
- ☐ Each site has been trained
- ☐ Each coordinator has logged in and changed their password
- ☐ Escalation path documented and shared with sites (coordinator → content author → you → PI)

Trial-specific

- ☐ IRB approval references the deployed CRF version
- ☐ Sponsor / DCC sign-off recorded

When every box is checked, schedule a 30-minute go-live call with the content author and the site PIs. On the call: confirm the URL works for everyone, confirm the credentials, ask each site PI for go/no-go, and switch the trial banner to *Live*.

Chapter 12. Day-2 operations

The first two weeks after go-live are the highest-risk window. After that, the operational rhythm settles.

Daily (week 1)

- Morning: check container health (`docker compose ps`).
- Morning: check overnight backup (`ls -la backups/ | head`).
- Throughout the day: respond to escalations from the content author within 1 hour.

- Evening: spot-check the application URL is reachable.

Weekly (week 2 onward)

- Run `verify_audit_log_integrity()` (§10.1). Log result.
- Check disk space on the VM (`df -h`).
- Check backup retention (most recent 30 days present).
- Review application logs for errors (`docker compose logs zzedc`).

Monthly

- Review user roster with content author for stale accounts.
- Review the validation-rule list for additions / removals the trial has accumulated.
- Sample-export one subject's data and confirm it round-trips through the *Export* tab cleanly.

Quarterly

- Restore drill (§9.3).
- Key rotation review (rotate annually, but check quarterly for compromise indicators).
- Update R and zzedc to the latest patch versions; re-run the full test suite (`Rscript -e 'tinytest::run_test_dir(...)'`) before pushing to production.

When to call the project maintainer

Open a GitHub issue at the project's bug tracker for: chain breaks, application crashes, data corruption, security findings. Include the redacted log output and the package version. Do not include subject data.

Appendix A. Migrating from REDCap

If your trial is moving from REDCap, ZZedc has a one-call importer. The full guide is at `vignette('mysql-redcap-migration')`. The short version:

```
# REST API path (no MySQL access needed):
result <- import_redcap_to_zzedc_db(
  source      = 'api',
  api_url     = 'https://redcap.example.org/api/',
  api_token   = Sys.getenv('REDCAP_API_TOKEN'),
  db_path     = '/srv/zzedc/MEMORY-001/study.db',
  overwrite   = TRUE
)

result$success                # TRUE
result$audit_replay$completeness # full / partial / empty
result$audit_replay$marker_inserted # TRUE if not full
result$audit_replay$chain_validates # TRUE
length(result$skipped_rules)    # rules to manually port

# Live MySQL path (faster, audit log always full):
library(RMariaDB)
conn <- DBI::dbConnect(
  MariaDB(),
  host      = 'redcap.example.org',
  dbname    = 'redcap',
```



```

user      = 'redcap_reader',
password = Sys.getenv('REDCAP_DB_PASSWORD'))

result <- import_redcap_to_zzedc_db(
  source    = 'db',
  conn      = conn,
  pid       = 42L,
  db_path   = '/srv/zzedc/MEMORY-001/study.db',
  overwrite = TRUE
)
DBI::dbDisconnect(conn)

```

After migration, share the resulting database with the content author and run their Chapter 4 dry-run protocol against the imported study.

Appendix B. Per-backend cheat sheets

Each backend has a dedicated vignette; this appendix is just a pointer with the one-line config snippet you would put in `config.yml`.

Backend	One-line config	Vignette
SQLite	<code>db_backend: sqlite +</code> <code>db_path: ...</code>	<code>quickstart-sqlite</code>
DuckDB	<code>db_backend: duckdb +</code> <code>db_path: ...</code>	<code>quickstart-duckdb</code>
PostgreSQL	<code>db_backend: postgresql +</code> <code>host/port/name/user</code>	<code>quickstart-postgresql</code>
MySQL/MariaDB	<code>db_backend: mysql +</code> <code>host/port/name/user</code>	<code>quickstart-mysql</code>
ClickHouse	<code>db_backend: clickhouse +</code> <code>host/port/name/user</code>	<code>quickstart-clickhouse</code>

Appendix C. Where the regulatory claims live in the code

If you are responding to an audit, here is a map from claim to file:

Claim	Implemented in
Encryption at rest (AES-256)	<code>R/encryption_utils.R</code> , <code>R/db_connection.R</code>
Hash-chained audit log	<code>R/audit_logging.R</code> , <code>R/cfr_part11_extensions.R</code>
Audit-log integrity validation	<code>R/audit_logging.R::verify_audit_log_integrity()</code>
Electronic signatures (21 CFR Part 11)	<code>R/electronic_signatures.R</code>
GDPR data-subject rights	<code>R/dsar.R</code> , <code>R/erasure.R</code> , <code>R/portability.R</code> , <code>R/rectification.R</code>
GDPR consent management	<code>R/consent.R</code>
Data retention enforcement	<code>R/retention.R</code>
Breach notification	<code>R/breach_notification.R</code>
Privacy impact assessment	<code>R/privacy_impact.R</code>

Claim	Implemented in
REDCap migration with audit replay	R/redcap_import.R
Multi-backend portability	R/portability.R, R/db_adapter_*.R

Companion guides and references

- Content-Author Guide — the parallel guide for the non-technical team member
- Encryption at Rest — detailed encryption architecture
- Disaster Recovery — full DR scenarios and runbooks
- AWS / DevOps Quick Start — cloud deployment patterns
- REDCap Migration Roadmap
- Demonstration Trial Setup — a worked example with synthetic data
- Advanced Features — custom modules, REST API, integration examples